

# WEEK5 Task2

## 사용자 정의 손실함수란..?

사용자 정의 손실 함수(Custom Loss Function)란, 딥러닝 모델에서 사용자가 특정한 문제에 맞는 손실 함수를 직접 정의하여 학습에 활용하는 것을 말함.

딥러닝 프레임워크(ex. TensorFlow, PyTorch 등)는 일반적으로 많이 사용하는 손실 함수(예: MSE, Cross-Entropy)를 제공하지만, 특정 상황에서는 기존 손실 함수로는 적합하지 않은 문제를 해결해야 할 때가 있음.

이럴 때 사용자 정의 손실 함수를 만들어 적용할 수 있다. -> task2에서는 후버 손실함수를 직접 수식을 만들어서 사용

## 사용자 정의 층이란..?

사용자 정의 층(Custom Layer)이란, 딥러닝 모델의 구성 요소인 층(Layer)을 사용자가 직접 정의하고 설계하는 것. 딥러닝 프레임워크는 일반적으로 자주 사용하는 층(Dense, LSTM 등)을 제공하지만, 특정 문제를 해결하거나 특별한 계산 방식을 적용해야 할 때는 표준 층만으로는 충분하지 않을 수 있다.

이럴 때 사용자 정의 층을 만들어 원하는 동작을 구현할 수 있다.

\_\_init\_\_:

- 층에서 사용될 설정값이나 초기 파라미터를 정의.
- 학습 가능한 변수(가중치, 편향 등)를 만들지 않음.
- 예: 출력 뉴런의 개수, 활성화함수 설정 등.

build:

- 층의 학습 가능한 변수를 정의.
- 입력 데이터의 크기에 따라 동적으로 변수(가중치, 편향)를 생성.

call:

- 층의 주요 연산 로직을 구현.
- 입력 데이터를 받아 변환된 출력을 반환함.

```
# 데이터 준비
```

```
X, y = make_regression(n_samples=1000, n_features=10, noise=0.1)
```

```
y = y.reshape(-1, 1)
```

- make\_regression 데이터셋을 사용.
- n\_samples=1000: 샘플 수 1000개.
- n\_features=10: 특징 10개.
- noise=0.1: 데이터에 0.1 크기의 노이즈 추가.
- y.reshape(-1, 1): 레이블의 형태 변환(2차원)

- 데이터셋: sklearn.datasets 에서 제공하는 make\_regression 데이터를 사용합니다.
- 데이터 처리:
  - i. 훈련 데이터(80%) 와 테스트 데이터(20%) 로 분리하세요.
  - ii. 입력 데이터를 표준화하여 평균 0, 표준편차 1로 변환하세요.

```
# 훈련 데이터(80%) 와 테스트 데이터(20%) 분리
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- 데이터를 훈련 세트(80%)와 테스트 세트(20%)로 분리.

```
# 입력 데이터를 표준화
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)
y_train = scaler_y.fit_transform(y_train)
y_test = scaler_y.transform(y_test)
```

- `MyDenseLayer` 라는 사용자 정의 층을 만드세요.
- 요구사항:
  - i. 입력 크기에 따라 가중치(**weight**)와 편향(**bias**)를 생성하고 학습 가능하도록 설정합니다.
  - ii. 입력 데이터를 받아 **ReLU** 활성화 함수를 적용합니다.

StandardScaler: 데이터를 표준화하여 평균 0, 표준편차 1로 변환.

- `fit_transform`: 훈련 세트의 평균과 표준편차로 데이터를 변환.
- `transform`: 테스트 세트는 훈련 세트의 기준에 맞춰 변환.

```
class MyDenseLayer(Layer):
    def __init__(self, units, **kwargs):
        super(MyDenseLayer, self).__init__(**kwargs)
        self.units = units
```

- `MyDenseLayer`: 사용자 정의 층 클래스.
- `units`: 출력 뉴런 수.

```
def build(self, input_shape):
    self.weight = self.add_weight(shape=(input_shape[-1], self.units),
                                   initializer='random_normal',
                                   trainable=True, name='weight')
    self.bias = self.add_weight(shape=(self.units,),
                                 initializer='zeros',
                                 trainable=True, name='bias')
```

build: 입력 형태에 따라 가중치(weight)와 편향(bias)을 정의.

- shape: 가중치의 크기는 (입력 차원, 출력 뉴런 수).
- initializer: 가중치는 정규분포(random\_normal)로 초기화, 편향은 0으로 초기화.
- trainable=True: 학습 가능한 파라미터로 설정.

```
def call(self, inputs):
    z = tf.matmul(inputs, self.weight) + self.bias
    return tf.nn.relu(z)
```

- call: 주요 연산 로직 구현.
- tf.matmul: 입력과 가중치를 행렬 곱.
- ReLU 활성화함수 적용.



# 사용자 정의 손실 함수 (후버 손실)

```
def huber_loss(y_true, y_pred):  
    delta = 1.0  
    error = y_true - y_pred  
    is_small_error = tf.abs(error) <= delta  
    small_error_loss = 0.5 * tf.square(error)  
    big_error_loss = delta * (tf.abs(error) - 0.5 * delta)  
    return tf.where(is_small_error, small_error_loss, big_error_loss)
```

- 후버 손실(Huber Loss) 를 구현하세요.
- 요구사항:
  - i. 임계값( `delta` )은 1로 설정합니다.
  - ii. 작은 오차는 제곱 손실, 큰 오차는 선형 손실로 처리합니다.

후버 손실 함수:

- delta: 경계값(작은 오차와 큰 오차 구분).
- 작은 오차는 MSE( $0.5 * \text{error}^2$ ), 큰 오차는 MAE( $\text{delta} * \text{error} - 0.5 * \text{delta}^2$ )로 계산.

# 모델 설계

```
model = Sequential([  
    MyDenseLayer(32, name='hidden1'),  
    MyDenseLayer(32, name='hidden2'),  
    tf.keras.layers.Dense(1, name='output') # 출력층  
)
```

- 사용자 정의 층과 후버 손실 함수를 사용하여 신경망을 설계합니다.
- 요구사항:
  - i. 구조: 2개의 은닉층(각 32개의 뉴런)과 1개의 출력층.
  - ii. **Optimizer:** Adam.
  - iii. 평가지표: MSE (Mean Squared Error).
  - iv. 훈련: 10 epoch, batch size=32.

신경망 모델 생성:

- 은닉층 2개(MyDenseLayer 사용, 각 32개 뉴런).
- 출력층: Dense(1)로 출력값 1개 생성.

# 모델 컴파일

```
model.compile(optimizer=Adam(),  
              loss=huber_loss,  
              metrics=['mse'])
```

모델 컴파일:

- optimizer=Adam: 옵티마이저로 Adam 사용.
- loss=huber\_loss: 앞에서 정의한 후버 손실 함수 적용.
- metrics=['mse']: 모델 평가에 MSE 사용.

# 모델 훈련

```
history = model.fit(X_train, y_train,  
                   epochs=10,  
                   batch_size=32,  
                   validation_split=0.2,  
                   verbose=1)
```

모델 학습:

- epochs=10: 학습 반복 10회.
- batch\_size=32: 배치 크기 32.
- validation\_split=0.2: 훈련 데이터의 20%를 검증 데이터로 사용.
- verbose=1: 훈련 중 손실값 출력.



```
# 모델 평가
test_loss, test_mse = model.evaluate(X_test, y_test, verbose=0)
print(f"테스트 데이터에서의 MSE: {test_mse:.4f}")
```

```
# 첫 번째 샘플의 예측값과 실제값 출력
first_sample_pred = scaler_y.inverse_transform(model.predict(X_test[:1]))
first_sample_true = scaler_y.inverse_transform(y_test[:1])

print(f"첫 번째 샘플의 예측값: {first_sample_pred[0][0]:.4f}")
print(f"첫 번째 샘플의 실제값: {first_sample_true[0][0]:.4f}")
```

- model.predict: 첫 번째 테스트 샘플에 대한 예측값.
- scaler\_y.inverse\_transform: 원래 스케일로 복원.
- 첫 번째 샘플의 예측값과 실제값 출력.

1. 모델 훈련 중 손실값 출력:
2. 테스트 데이터에서의 MSE 출력:
3. 첫 번째 샘플의 예측값과 실제값 출력:

- 요구사항:
  - i. 테스트 데이터에서 **MSE**를 출력하세요.
  - ii. 테스트 데이터 중 첫 번째 샘플의 **예측값**과 **실제값**을 출력하세요.

```
Epoch 1/10
20/20 _____ 3s 28ms/step - loss: 0.4407 - mse: 1.0524 - val_loss: 0.3364 - val_mse: 0.7469
Epoch 2/10
20/20 _____ 1s 10ms/step - loss: 0.3465 - mse: 0.7820 - val_loss: 0.2109 - val_mse: 0.4397
Epoch 3/10
20/20 _____ 0s 12ms/step - loss: 0.2131 - mse: 0.4659 - val_loss: 0.0423 - val_mse: 0.0846
Epoch 4/10
20/20 _____ 1s 7ms/step - loss: 0.0415 - mse: 0.0830 - val_loss: 0.0146 - val_mse: 0.0291
Epoch 5/10
20/20 _____ 0s 8ms/step - loss: 0.0135 - mse: 0.0270 - val_loss: 0.0050 - val_mse: 0.0099
Epoch 6/10
20/20 _____ 0s 10ms/step - loss: 0.0062 - mse: 0.0125 - val_loss: 0.0030 - val_mse: 0.0060
Epoch 7/10
20/20 _____ 1s 10ms/step - loss: 0.0037 - mse: 0.0075 - val_loss: 0.0022 - val_mse: 0.0044
Epoch 8/10
20/20 _____ 0s 10ms/step - loss: 0.0027 - mse: 0.0054 - val_loss: 0.0020 - val_mse: 0.0040
Epoch 9/10
20/20 _____ 0s 13ms/step - loss: 0.0021 - mse: 0.0042 - val_loss: 0.0018 - val_mse: 0.0035
Epoch 10/10
20/20 _____ 1s 11ms/step - loss: 0.0019 - mse: 0.0038 - val_loss: 0.0017 - val_mse: 0.0033
테스트 데이터에서의 MSE: 0.0039
1/1 _____ 0s 200ms/step
첫 번째 샘플의 예측값: 140.5307
첫 번째 샘플의 실제값: 131.5677
```